Introduction to Haskell Programming

Polymorphism and type classes

S P Suresh

Chennai Mathematical Institute

Haskell @ CMI

Functions and types

- Consider these definitions

```
myLength []       = 0
myLength (x:xs)   = 1 + myLength xs

myReverse []      = []
myReverse (x:xs) = myReverse xs ++ [x]

myInit [x]        = []
myInit (x:xs)     = x:myInit xs
```

Functions and types

- Consider these definitions

```
myLength []       = 0
myLength (x:xs)   = 1 + myLength xs

myReverse []      = []
myReverse (x:xs) = myReverse xs ++ [x]

myInit [x]        = []
myInit (x:xs)     = x:myInit xs
```

- None of these functions look into the elements of the list

Functions and types

- Consider these definitions

```
myLength []      = 0
myLength (x:xs)  = 1 + myLength xs

myReverse []     = []
myReverse (x:xs) = myReverse xs ++ [x]

myInit [x]       = []
myInit (x:xs)    = x:myInit xs
```

- None of these functions look into the elements of the list
- Will work over lists of any type!

Polymorphism

- Functions that work across multiple types

Polymorphism

- Functions that work across multiple types
- Use type variables to denote flexibility

Polymorphism

- Functions that work across multiple types
- Use type variables to denote flexibility
- a, b, c are place holders for types

Polymorphism

- Functions that work across multiple types
- Use type variables to denote flexibility
- `a`, `b`, `c` are place holders for types
- `[a]` is a list of elements of type `a`

Polymorphism

- Functions that work across multiple types
- Use type variables to denote flexibility
- `a`, `b`, `c` are place holders for types
- `[a]` is a list of elements of type `a`
- **Example functions**:

```
head    :: [a] → a
length  :: [a] → Int
reverse :: [a] → [a]
take    :: Int → [a] → [a]
```

Polymorphism

- Functions that work across multiple types
- Use type variables to denote flexibility
- `a`, `b`, `c` are place holders for types
- `[a]` is a list of elements of type `a`
- **Example functions**:

```
head    :: [a] → a
length  :: [a] → Int
reverse :: [a] → [a]
take    :: Int → [a] → [a]
```

- All `a`'s in the type should be instantiated in the same way

Sorting?

- `sort` is a built-in function that re-arranges a list in ascending order

Sorting?

- `sort` is a built-in function that re-arranges a list in ascending order
- Can we write `[a] → [a]` as the type for `sort`?

Sorting?

- `sort` is a built-in function that re-arranges a list in ascending order
- Can we write `[a] → [a]` as the type for `sort`?
- Can we sort any type of list?

Sorting?

- **`sort`** is a built-in function that re-arranges a list in ascending order
- Can we write `[a] → [a]` as the type for **`sort`**?
- Can we sort any type of list?
- What about a list of functions?

```
[factorial, (+3), (*5)] :: [Int → Int]
```

Sorting?

- `sort` is a built-in function that re-arranges a list in ascending order
- Can we write `[a] → [a]` as the type for `sort`?
- Can we sort any type of list?
- What about a list of functions?

```
[factorial, (+3), (*5)] :: [Int → Int]
```

- How do we compare `f < g` for functions?

Type classes

- Want to assign a type as follows:

Type classes

- Want to assign a type as follows:
- `sort :: [a] → [a]` provided we can compare values of type `a`

Type classes

- Want to assign a type as follows:
- `sort :: [a] → [a]` provided we can compare values of type `a`
- A **type class** is a collection of types with a required property

Type classes

- Want to assign a type as follows:
- `sort :: [a] → [a]` provided we can compare values of type `a`
- A **type class** is a collection of types with a required property
- The type class `Ord` contains all types whose values can be compared

Type classes

- `Ord t` is a predicate that evaluates to `True` if `t` belongs to type class `Ord`

Type classes

- `Ord t` is a predicate that evaluates to `True` if `t` belongs to type class `Ord`
- **Terminology**: There is an `Ord` instance of type `t`

Type classes

- `Ord t` is a predicate that evaluates to `True` if `t` belongs to type class `Ord`
- **Terminology**: There is an `Ord` instance of type `t`
- **Alternatively**: `t` is an instance of `Ord`

Type classes

- `Ord t` is a predicate that evaluates to `True` if `t` belongs to type class `Ord`
- **Terminology**: There is an `Ord` instance of type `t`
- **Alternatively**: `t` is an instance of `Ord`
- If `t` is an instance of `Ord`, then $<$, $\leqslant$, $>$, $\geqslant$, $=\!=$, $/\!=$ are defined for `t`

Type classes

- `Ord t` is a predicate that evaluates to `True` if `t` belongs to type class `Ord`
- **Terminology**: There is an `Ord` instance of type `t`
- **Alternatively**: `t` is an instance of `Ord`
- If `t` is an instance of `Ord`, then $<$, $\leqslant$, $>$, $\geqslant$, $=\!=$, $/\!=$ are defined for `t`
- For `t` to be an instance of `Ord`, it should also be an instance of `Eq`

Type classes

- If `t` is an instance of **`Eq`**, then `==` and `/=` are defined for `t`

Type classes

- If `t` is an instance of **`Eq`**, then $=$ and $\neq$ are defined for `t`
- If `t` is an instance of **`Ord`**, then $<$, $\leqslant$, $>$, $\geqslant$ are also defined for `t` (in addition to $=$ and $\neq$)

Type classes

- If `t` is an instance of **`Eq`**, then $=$ and $\neq$ are defined for `t`
- If `t` is an instance of **`Ord`**, then $<$, $\leqslant$, $>$, $\geqslant$ are also defined for `t` (in addition to $=$ and $\neq$)
- Back to sorting ...

Type classes

- If `t` is an instance of **`Eq`**, then $=$ and $\neq$ are defined for `t`
- If `t` is an instance of **`Ord`**, then $<$, $\leqslant$, $>$, $\geqslant$ are also defined for `t` (in addition to $=$ and $\neq$)
- Back to sorting ...
- The correct typing uses a **class constraint**:

```
sort :: Ord a ⇒ [a] → [a]
```

Type classes

- If `t` is an instance of **`Eq`**, then $=$ and $\neq$ are defined for `t`
- If `t` is an instance of **`Ord`**, then $<$, $\leqslant$, $>$, $\geqslant$ are also defined for `t` (in addition to $=$ and $\neq$)
- Back to sorting ...
- The correct typing uses a **class constraint**:

```
sort :: Ord a ⇒ [a] → [a]
```

- If `a` is an instance of **`Ord`**, **`sort`** is of type `[a] → [a]`

Typing `elem`

- How can we type `elem`?

```
elem x []       = False
elem x (y:ys)   = x == y || elem x ys
```

Typing `elem`

- How can we type `elem`?

```
elem x []       = False
elem x (y:ys)   = x == y || elem x ys
```

- Consider the list

```
funclist = [factorial, (+3), (*5)] :: [Int → Int]
```

Typing `elem`

- How can we type `elem`?

```
elem x []       = False
elem x (y:ys)   = x == y || elem x ys
```

- Consider the list

```
funclist = [factorial, (+3), (*5)] :: [Int → Int]
```

- How to evaluate `elem f funclist`?

Equality

- Can we check `f == g` for functions?

Equality

- Can we check `f == g` for functions?
- `f x == g x` for all `x`?

Equality

- Can we check `f == g` for functions?
- `f x == g x` for all `x`?
- Recall that `f x` may not terminate

Equality

- Can we check `f == g` for functions?
- `f x == g x` for all `x`?
- Recall that `f x` may not terminate
- For instance:

```
factorial 0 = 1
factorial n = n * factorial (n-1)
```

Equality

- Can we check `f == g` for functions?
- `f x == g x` for all `x`?
- Recall that `f x` may not terminate
- For instance:

```
factorial 0 = 1
factorial n = n * factorial (n-1)
```

- `factorial (-1)` does not terminate

Equality

- Can we check `f == g` for functions?
- `f x == g x` for all `x`?
- Recall that `f x` may not terminate
- For instance:

```
factorial 0 = 1
factorial n = n * factorial (n-1)
```

- `factorial (-1)` does not terminate
- `f == g` implies that for all `x`, `f x` terminates iff `g x` does

Equality

- Can we check `f == g` for functions?
- `f x == g x` for all `x`?
- Recall that `f x` may not terminate
- For instance:

```
factorial 0 = 1
factorial n = n * factorial (n-1)
```

- `factorial (-1)` does not terminate
- `f == g` implies that for all `x`, `f x` terminates iff `g x` does
- **Conclusion:** Not all types admit an equality check!

The type class `Eq`

- `Eq a` holds if $=$, $\neq$ are defined on `a`

The type class `Eq`

- `Eq a` holds if $=$, $\neq$ are defined on `a`
- The typing for `elem` is:

```
elem :: Eq a ⇒ a → [a] → Bool
```

The type class `Eq`

- `Eq a` holds if $=$, $\neq$ are defined on `a`
- The typing for `elem` is:

```
elem :: Eq a ⇒ a → [a] → Bool
```

- If `Eq a` and `Eq b`, then `Eq (a,b)`, `Eq [a]`, `Eq [[a]]`, ...

The type class `Eq`

- `Eq a` holds if $=$, $\neq$ are defined on `a`
- The typing for `elem` is:

```
elem :: Eq a ⇒ a → [a] → Bool
```

- If `Eq a` and `Eq b`, then `Eq (a,b)`, `Eq [a]`, `Eq [[a]]`, ...
- But we cannot extend `Eq a`, `Eq b` to `Eq (a → b)`

The type class `Ord`

- If `Ord a` holds then $<$, $\leqslant$, $>$, $\geqslant$, $==$, $/=$ are defined for `a`

The type class `Ord`

- If `Ord a` holds then $<$, $\leqslant$, $>$, $\geqslant$, $=\!=$, $/\!=$ are defined for `a`
- Also defined are `min :: a → a → a` and `max :: a → a → a`

The type class `Ord`

- If `Ord a` holds then $<$, $\leqslant$, $>$, $\geqslant$, $=\!=$, $/\!=$ are defined for `a`
- Also defined are `min :: a → a → a` and `max :: a → a → a`
- If `Ord a` then `Ord [a]` – lexicographic (dictionary) order

The type class `Ord`

- If `Ord a` holds then $<$, $\leqslant$, $>$, $\geqslant$, $=\!=$, $/\!=$ are defined for `a`
- Also defined are `min :: a → a → a` and `max :: a → a → a`
- If `Ord a` then `Ord [a]` – lexicographic (dictionary) order
- If `Ord a` and `Ord b`, then `Ord (a,b)`

The type class `Ord`

- If `Ord a` holds then $<$, $\leqslant$, $>$, $\geqslant$, $=\!=$, $/\!=$ are defined for `a`
- Also defined are `min :: a → a → a` and `max :: a → a → a`
- If `Ord a` then `Ord [a]` – lexicographic (dictionary) order
- If `Ord a` and `Ord b`, then `Ord (a,b)`
- Cannot extend `Ord a`, `Ord b` to `Ord (a → b)`

The type class Num

- Recall the function sum

```
sum []     = 0
sum (x:xs) = x + (sum xs)
```

The type class `Num`

- Recall the function `sum`

```
sum []     = 0
sum (x:xs) = x + (sum xs)
```

- `sum` requires `+` to be defined on list elements

The type class `Num`

- Recall the function `sum`

```
sum []     = 0
sum (x:xs) = x + (sum xs)
```

- `sum` requires `+` to be defined on list elements
- `Num a` says `a` is a number, and supports basic arithmetic operations

The type class Num

- Recall the function sum

```
sum []     = 0
sum (x:xs) = x + (sum xs)
```

- sum requires + to be defined on list elements
- Num a says a is a number, and supports basic arithmetic operations
- The correct typing for sum

```
sum :: (Num a) ⇒ [a] → a
```

The type class `Num`

- Any type `a` belonging to the class `Num` has the following definitions:

```
(+)         :: a → a → a
(-)         :: a → a → a
(*)         :: a → a → a
negate      :: a → a
abs         :: a → a
signum      :: a → a
fromInteger :: Integer → a
```

The type class `Num`

- Any type `a` belonging to the class `Num` has the following definitions:

```
(+)         :: a → a → a
(-)         :: a → a → a
(*)         :: a → a → a
negate      :: a → a
abs         :: a → a
signum      :: a → a
fromInteger :: Integer → a
```

- **No division operation**

The type class `Num`

- Any type `a` belonging to the class `Num` has the following definitions:

```
(+)         :: a → a → a
(-)         :: a → a → a
(*)         :: a → a → a
negate      :: a → a
abs         :: a → a
signum      :: a → a
fromInteger :: Integer → a
```

- **No division operation**
- `signum` tells if the input is negative (`-1`) or zero (`0`) or positive (`1`)

The type class `Integral`

- Numerical types whose values are integers

The type class `Integral`

- Numerical types whose values are integers
- Supports integer division and remainder

The type class `Integral`

- Numerical types whose values are integers
- Supports integer division and remainder
- Any type `a` belonging to the class `Integral` has the following definitions (in addition to the `Num` functions):

```
div         :: a → a → a
mod         :: a → a → a
quot        :: a → a → a
rem         :: a → a → a
divMod      :: a → a → (a,a)
quotRem     :: a → a → (a,a)
toInteger   :: a → Integer
```

The type class `Fractional`

- Numerical types whose values are non-integral

The type class `Fractional`

- Numerical types whose values are non-integral
- Supports fractional division and reciprocation

The type class `Fractional`

- Numerical types whose values are non-integral
- Supports fractional division and reciprocation
- Any type `a` belonging to the class `Fractional` has the following definitions (in addition to the `Num` functions):

```
(/)             :: a → a → a
recip           :: a → a
fromRational    :: Rational → a
```

The class Show

- `Show` – types whose values can be displayed

The class Show

- `Show` – types whose values can be displayed
- For a type `t` to be an instance of `Show`, we need a definition for the following function:

```
show :: a → String
```

The class Show

- `Show` – types whose values can be displayed
- For a type `t` to be an instance of `Show`, we need a definition for the following function:

```
show :: a → String
```

- Provides a printable representation for values of type `a`

The class Show

- `Show` – types whose values can be displayed
- For a type `t` to be an instance of `Show`, we need a definition for the following function:

```
show :: a → String
```

- Provides a printable representation for values of type `a`
- The built-in datatypes are all instances of the expected type classes

The class Show

```
>>> show False
          "False"
>>> show 'a'
          "'a'"
>>> show [0..9]
          "[0,1,2,3,4,5,6,7,8,9]"
>>> show ('a', False)
          "('a',False)"
>>> show (+)
          No instance for ‘Show (Integer → Integer → Integer)’
```

The class `Read`

- `Read` – types whose values can be read

The class `Read`

- `Read` – types whose values can be read
- For a type `t` to be an instance of `Read`, we need a definition for the following function:

```
read :: String → a
```

The class `Read`

- `Read` – types whose values can be read
- For a type `t` to be an instance of `Read`, we need a definition for the following function:

```
read :: String → a
```

- When using the `read` function, we need to specify the target type ...

The class `Read`

- `Read` – types whose values can be read
- For a type `t` to be an instance of `Read`, we need a definition for the following function:

```
read :: String → a
```

- When using the `read` function, we need to specify the target type ...
- unless the surrounding context fixes the type

The class `Read`

- `Read` – types whose values can be read
- For a type `t` to be an instance of `Read`, we need a definition for the following function:

```
read :: String → a
```

- When using the `read` function, we need to specify the target type ...
- unless the surrounding context fixes the type
- The built-in datatypes are all instances of the expected type classes

The class Read

```
>>> read "[0,1,2,3,4]" :: [Integer]
        [0,1,2,3,4]
>>> read "('a', False)" :: (Char, Bool)
        ('a',False)
>>> read "False" :: Bool
        False
>>> read "False"
        *** Exception: Prelude.read: no parse
>>> not (read "False")
        True
>>> not (read "abc")
        *** Exception: Prelude.read: no parse
```